RTL Design Sherpa

Timing Characterization -- Synthesis Guide Per-FUB recipes, the multi-flow SDC, and how to read the results

September 29, 2026

Table of Contents

Timing Characterization – Synthesis Guide....6
1. What This Design Is and Why It Exists....7
2. Architecture Overview....8
3. FUB Catalog – Characterization Purpose of Each Block....10
3.1 nand_chain (NAND Gate Binary Tree)....10
3.2 inverter_chain (Inverter Chain)....10
3.3 xor_tree (XOR Gate Binary Tree)....11
3.4 carry_chain (Ripple-Carry Adder)....11
3.5 multiplier_tree (Multiplier Wrapper)....12
3.6 mux_tree (Binary Mux Tree)....13
3.7 queue_depth (FIFO Queue)....13
3.8 clock_divider_chain (Clock Divider Cascade)....14
3.9 gray_counter_chain (Gray Counter)....14
4. The LFSR – Anti-Optimization Engine....16
5. SDC Constraints Strategy....17
5.1 Multi-Flow SDC....17
5.2 Overridable Parameters....17
5.3 What Each Flow Requires Externally....17
5.4 The 80/20 I/O Split....18
5.5 Sweeping Frequency....19
5.6 Tool-Specific Features....19
6. Synthesis Workflow....20

6.1 Single-FUB Characterization....20
6.2 Parameter Sweep....20
6.3 Cross-FUB Comparison....20
6.4 Cross-Technology Comparison....20
7. Interpreting Results....22
7.1 Key Metrics....22
7.2 Expected Trends....22
7.3 Normalizing Across Technologies....23
8. File Organization....24
9. Quick Reference – Recommended First Runs....25

List of Figures

No figures in this document.

List of Tables

No tables in this document.

Timing Characterization – Synthesis Guide

Version: 1.1 **Last Updated:** 2026-10-20 **Purpose:** Detailed guide to synthesizing and interpreting timing characterization results

---

1. What This Design Is and Why It Exists

`char_top` is a **synthesis benchmarking harness**. It does not implement a useful datapath – its sole purpose is to force synthesis tools to build specific combinational structures between registered endpoints so that the resulting timing reports can be compared across:

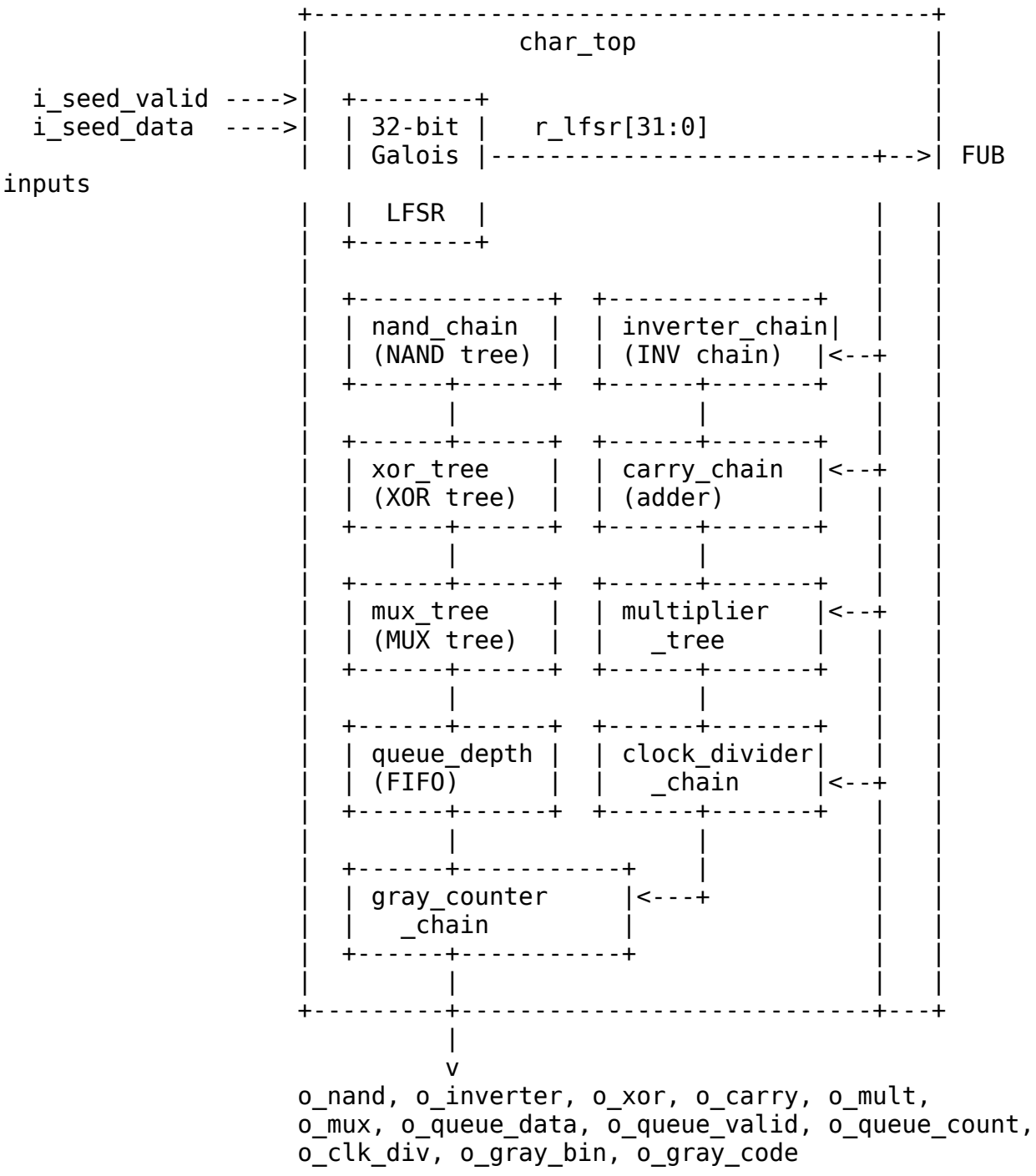
- FPGA families (Xilinx 7-series vs. UltraScale vs. Cyclone/Stratix)
- ASIC standard-cell libraries (TSMC, GF, Samsung at various nodes)
- Synthesis tool settings (effort levels, mapping strategies)
- Parameter sweeps (wider adders, deeper trees, bigger multipliers)

Every flip-flop input is driven by a 32-bit Galois LFSR that produces a maximal-length pseudo-random sequence. Every flip-flop output is routed to a top-level port. Together these two measures guarantee that the synthesis tool **cannot** constant-propagate inputs or prune unused outputs. Additionally, every internal combinational net carries vendor-appropriate `dont_touch` / `preserve` attributes as a second line of defense.

The design is organized as a single top-level wrapper (`char_top`) that conditionally instantiates nine **Functional Unit Blocks (FUBs)**. Each FUB isolates a different class of combinational logic so that its timing can be reported independently.

---

2. Architecture Overview



Each FUB is controlled by an `EN_*` parameter. When disabled (`EN_*=0`), the corresponding generate block emits only a tie-to-zero assignment and consumes zero logic. This allows targeted synthesis runs that isolate a single FUB or any combination.

3. FUB Catalog – Characterization Purpose of Each Block

3.1 nand_chain (NAND Gate Binary Tree)

Attribute	Value
RTL file	`rtl/fub/nand_chain.sv`
Enable	`EN_NAND_TREE`
Key params	`NAND_LEVELS`, `NUM_FLOPS`
Critical path	LEVELS NAND gate delays (root to leaf)
What it measures	Raw 2-input NAND propagation through LUT fabric

Structure: A complete binary tree of 2-input NAND gates. The leaf level reads from registered input flops; the root feeds a registered output flop. Heap-style indexing maps tree nodes to a flat array. When `2^LEVELS > NUM_FLOPS`, leaf inputs reuse flops via modulo wrapping – this preserves the full tree depth (and critical path) while bounding flop count.

Synthesis goal: Measure how NAND gate depth maps to LUT levels on FPGA or gate levels on ASIC. A LEVELS=8 tree creates 255 NAND gates with an 8-gate critical path. Compare slack at 500 MHz across devices to understand per-gate delay.

3.2 inverter_chain (Inverter Chain)

Attribute	Value
RTL file	`rtl/fub/inverter_chain.sv`
Enable	`EN_INVERTER_CHAIN`
Key params	`INVERTER_COUNT`
Critical path	INVERTER_COUNT inverter delays (linear)
What it measures	Pure gate delay without fan-in effects

Structure: A simple linear chain of inverters: `r_input_flop -> ~() -> ~() -> ... -> ~() -> r_out_flop`. Even count = same polarity as input; odd count = inverted.

Synthesis goal: Baseline single-gate delay measurement. Inverters have zero fan-in overhead and minimal routing, so they represent the **best-case** per-gate delay. Compare against NAND/XOR to

isolate fan-in overhead. Sweep INVERTER_COUNT to find the linear region and the point where routing delay begins to dominate.

3.3 xor_tree (XOR Gate Binary Tree)

Attribute	Value
RTL file	`rtl/fub/xor_tree.sv`
Enable	`EN_XOR_TREE`
Key params	`XOR_LEVELS`, `NUM_FLOPS`
Critical path	LEVELS XOR gate delays
What it measures	XOR-specific LUT packing efficiency

Structure: Identical topology to nand_chain but with XOR gates instead of NAND. Same heap indexing and modulo flop reuse.

Synthesis goal: XOR gates stress FPGA LUTs differently than NAND. On Xilinx, a 6-input LUT can absorb multiple XOR levels, while NAND chains often pack fewer levels per LUT. Comparing XOR tree slack against NAND tree slack at the same depth reveals LUT packing efficiency for each gate type.

3.4 carry_chain (Ripple-Carry Adder)

Attribute	Value
RTL file	`rtl/fub/carry_chain.sv`
Enable	`EN_CARRY_CHAIN`
Key params	`CARRY_WIDTH`
Critical path	WIDTH carry-propagation delays
What it measures	Dedicated carry-chain resource speed

Structure: Two registered input vectors feed a simple + operator. The synthesis tool infers a ripple-carry adder and maps it to dedicated fast-carry resources (CARRY4/CARRY8 on Xilinx, ALM carry on Quartus targets). The output is WIDTH+1 bits (including carry-out).

Synthesis goal: Measure the speed of the dedicated carry chain versus generic LUT routing. On modern FPGAs, carry chains run at sub-100ps per bit – far faster than LUT-routed equivalents.

Sweep CARRY_WIDTH from 8 to 256+ to find the width at which carry propagation becomes the limiting factor at the target frequency. This directly informs max-width adder decisions for production datapaths.

3.5 multiplier_tree (Multiplier Wrapper)

Attribute	Value
RTL file	`rtl/fub/multiplier_tree.sv`
Enable	`EN_MULTIPLIER`
Key params	`MULT_WIDTH`, `MULT_TYPE`
Critical path	Multiplier reduction tree depth
What it measures	DSP vs. LUT-fabric multiplier timing

Structure: Two registered input vectors feed a multiplier that produces a 2*WIDTH output. MULT_TYPE selects the implementation:

MULT_TYPE	Implementation	Available widths
0	Inferred (* operator)	Any
1	Dadda tree (structural)	8, 16, 32
2	Wallace tree (structural)	8, 16, 32
3	Wallace tree CSA (structural)	8, 16, 32
4	Dadda 4:2 compressor (structural)	8, 11, 24

Unsupported width/type combinations fall back to inferred.

Synthesis goal: Compare inferred multiplier timing (where the tool decides DSP vs. LUT) against explicit structural implementations. On Xilinx, inferred * at 16-bit width will map to a single DSP48E2 slice; structural types force LUT-only implementation. The slack difference directly quantifies the DSP advantage. Sweep MULT_WIDTH to find the crossover point where the tool stops using DSPs and switches to LUTs.

3.6 mux_tree (Binary Mux Tree)

Attribute	Value
RTL file	`rtl/fub/mux_tree.sv`
Enable	`EN_MUX_TREE`
Key params	`MUX_LEVELS`, `NUM_FLOPS`
Critical path	LEVELS 2:1 mux delays
What it measures	Mux-to-LUT mapping and routing fabric

Structure: A binary tree of 2:1 multiplexers. Data inputs come from registered flops; select inputs come from a separate registered select vector. The tree reduces 2^LEVELS data inputs down to a single output bit through LEVELS mux stages.

Synthesis goal: Muxes map to LUTs differently than logic gates. A 6-input LUT can implement a 4:1 mux (2 levels) in a single cell. Compare mux tree slack against NAND/XOR tree slack at the same depth to understand mux-to-LUT packing. Also reveals routing congestion effects at large tree sizes since every mux level requires routing from two subtrees.

3.7 queue_depth (FIFO Queue)

Attribute	Value
RTL file	`rtl/fub/queue_depth.sv`
Enable	`EN_QUEUE_DEPTH`
Key params	`FIFO_DATA_WIDTH`, `FIFO_DEPTH`
Critical path	FIFO read path + output register
What it measures	Memory inference thresholds and FIFO timing

Structure: Wraps `gaxi_fifo_sync` (from rtl/common). LFSR data is continuously written when the FIFO has space; reads are always enabled. An output flop captures the read data.

Synthesis goal: Sweep DATA_WIDTH and DEPTH to find the boundaries where the synthesis tool transitions between memory implementations:

FPGA transition	Typical boundary
Distributed RAM (LUTRAM) -> BRAM	~64 entries or ~2Kb total
BRAM -> URAM (UltraScale+)	~4K entries or ~36Kb total

Each transition changes the read latency and routing characteristics. The timing report reveals the cost of each memory type at the target frequency.

3.8 clock_divider_chain (Clock Divider Cascade)

Attribute	Value
RTL file	`rtl/fub/clock_divider_chain.sv`
Enable	`EN_CLK_DIVIDER`
Key params	`CLK_DIV_STAGES`, `CLK_DIV_CW`, `CLK_DIV_PICKOFF`
Critical path	Counter increment + pickoff mux per stage
What it measures	Counter-based clock generation timing

Structure: NUM_STAGES cascaded `clock_divider` instances. Stage 0 divides the input clock; each subsequent stage divides the previous stage's output. Each stage contains a COUNTER_WIDTH-bit counter and a pickoff mux that selects the divided output.

Synthesis goal: Characterize the timing of counter-based clock generation. The pickoff point determines which counter bit drives the divided clock – higher pickoff points mean longer counter chains before the toggle. Sweep CLK_DIV_STAGES to measure cascaded divider overhead and CLK_DIV_CW to test counter width scaling.

3.9 gray_counter_chain (Gray Counter)

Attribute	Value
RTL file	`rtl/fub/gray_counter_chain.sv`
Enable	`EN_GRAY_COUNTER`
Key params	`GRAY_WIDTH`
Critical path	Binary counter carry + XOR tree for Gray conversion
What it measures	Binary-to-Gray conversion timing at width

Structure: Wraps `counter_bingray` (from rtl/common). The internal binary counter uses dedicated carry logic; the Gray conversion is `gray = bin ^ (bin >> 1)`, which creates a WIDTH-1 wide XOR tree. Output flops capture both binary and Gray values.

Synthesis goal: The interesting path is the XOR tree for Gray conversion. At narrow widths, the XOR tree fits in a few LUTs. At wide widths (64+), it becomes a significant combinational depth. Sweep GRAY_WIDTH to find the crossover where Gray conversion dominates the counter carry chain in the critical path.

---

4. The LFSR – Anti-Optimization Engine

The shared 32-bit Galois LFSR (polynomial `x^32 + x^22 + x^2 + x + 1`) serves three purposes:

1. **Prevents constant propagation** – Every FUB input changes every cycle with a pseudo-random pattern, so the synthesis tool cannot simplify logic based on constant input values.

2. **Prevents output pruning** – All LFSR-driven data eventually reaches a top-level output port, so no internal logic can be removed as unused.

3. **Provides deterministic seeding** – The `i_seed_valid` / `i_seed_data` interface allows testbenches to load a known seed and predict the exact LFSR sequence, enabling cycle-accurate functional verification.

The RTL implementation:

```
r_lfsr <= {r_lfsr[30:0], 1'b0} ^ (r_lfsr[31] ? 32'h0040_0007 : 32'h0);
```

Each FUB's inputs are wired as combinational taps from `r_lfsr` with bit-modulo wrapping to fill widths wider than 32 bits. For two-input FUBs (carry, multiplier), the second operand uses a 16-bit-shifted mapping (`r_lfsr[(b+16) % 32]`) to ensure the two inputs are decorrelated.

---

5. SDC Constraints Strategy

5.1 Multi-Flow SDC

`char_top.sdc` supports three synthesis flows via the `FLOW` variable:

Flow	Tool	Set before sourcing
`"asic"`	Synopsys DC / Cadence Genus	Target library, operating conditions
`"vivado"`	Xilinx Vivado	Part number (e.g., `xc7a200t...`)
`"quartus"`	Quartus Prime	FAMILY, DEVICE in .qsf (`fpga/quartus/sweep_config.example.tcl`)

All overridable parameters use `if {![info exists ...]}` guards so you can set them in your synthesis script **before** sourcing the SDC:

```
# Example: Vivado flow at 300 MHz
set FLOW "vivado"
set TARGET_FREQ_MHZ 300.0
source char_top.sdc
```

5.2 Overridable Parameters

Parameter	Default	Rationale
`FLOW`	`"asic"`	Selects tool-specific constraint blocks
`TARGET_FREQ_MHZ`	500	Aggressive enough to expose path differences
`INPUT_DELAY_FRACTION`	0.80	Models heavy external source
`OUTPUT_DELAY_FRACTION`	0.20	Models light external sink
`CLK_UNCERTAINTY_NS`	0.100	Covers jitter + skew

5.3 What Each Flow Requires Externally

ASIC (Synopsys DC):

```
set target_library "your_stdcell.db"
set link_library   "* your_stdcell.db"
set_operating_conditions -max WC_COM
```

```
set FLOW "asic"
source char_top.sdc
```

ASIC (Cadence Genus):

```
set_db init_lib_search_path /path/to/libs
read_libs your_stdcell.lib
set FLOW "asic"
read_sdc char_top.sdc
```

Vivado:

```
set_property part xc7a200tffg1156-2 [current_project]
set FLOW "vivado"
read_xdc char_top.sdc
synth_design -top char_top -generic EN_CARRY_CHAIN=1 ...
```

Quartus:

```
# In .qsf:
set_global_assignment -name FAMILY "Cyclone V"
set_global_assignment -name DEVICE 5CGXFC5C6F27C7
set_global_assignment -name SDC_FILE timing_target.sdc   ;# a 6-line
wrapper, see below
set_global_assignment -name OPTIMIZATION_MODE "HIGH PERFORMANCE
EFFORT"
```

The Timing Analyzer reads SDC as Tcl, so the wrapper it is pointed at sets the flow and sources the master file (there is no “pre-SDC hook”):

```
# timing_target.sdc
set FLOW            quartus
set TARGET_FREQ_MHZ 200
source "/abs/path/to/rtl/syn/char_top.sdc"
```

This is exactly what `fpga/quartus/syn_sweep_quartus.tcl` generates per sweep point; `cd fpga && make quartus-sweep` runs the whole loop and `fpga/quartus/sta_reports.tcl` adds per-FUB slack / data delay / logic levels. Two Timing Analyzer differences are folded into the master SDC: `set_clock_uncertainty` takes `-to <clock>` with the value LAST, and `set_load` is not implemented (`read_sdc` aborts on it).

5.4 The 80/20 I/O Split

The 80% input delay / 20% output delay split is intentional:

- **Input delay (80%):** The LFSR register is the first stage. By consuming 80% of the period on the input side, we force the LFSR update to close with only 20% margin. Since the LFSR is a simple shift-XOR, it easily meets this constraint, and the “real” critical paths are the register-to-register paths within each FUB.

- **Output delay (20%):** FUB outputs go directly from registered output flops to top-level ports. The 20% output budget is trivially met for registered outputs.

The net effect is that **all interesting timing is in the register-to-register paths within each FUB**, which is exactly what we want to characterize.

5.5 Sweeping Frequency

To sweep the target frequency, override the `TARGET_FREQ_MHZ` variable before sourcing the SDC:

```
set TARGET_FREQ_MHZ 250.0
set FLOW "vivado"
source char_top.sdc
```

Typical sweep points: 100, 200, 300, 400, 500, 600, 750, 1000 MHz.

5.6 Tool-Specific Features

Feature	ASIC	Vivado	Quartus
Per-FUB path groups	`group_path` (active)	`report_timing -from` (manual)	`report_timing -from` (manual)
Dont-touch enforcement	`set_dont_touch` (active)	`DONT_TOUCH` property (commented)	`PRESERVE_REGISTER` (commented)
I/O driving model	`set_driving_cell` (commented, needs lib cell)	`IOSTANDARD` (commented)	Via .qsf `IO_STANDARD`
Output load	Via library	`set_load 5.0`	not supported by the Timing Analyzer; via .qsf `IO_STANDARD`
Per-FUB slack/delay/levels	`report_timing -group`	manual	`fpga/quartus/sta_reports.tcl` -> `fub_slack.csv` (automated)

The ASIC flow enables path groups and dont-touch by default since these are standard DC/Genus commands. Vivado and Quartus equivalents are commented out because they use property-based syntax that depends on the specific target; uncomment as needed.

---

6. Synthesis Workflow

6.1 Single-FUB Characterization

Enable only the FUB of interest to minimize compile time:

```
# Vivado example: synthesize only carry_chain
synth_design -top char_top \
    -generic EN_NAND_TREE=0 \
    -generic EN_INVERTER_CHAIN=0 \
    -generic EN_XOR_TREE=0 \
    -generic EN_CARRY_CHAIN=1 \
    -generic EN_MULTIPLIER=0 \
    -generic EN_MUX_TREE=0 \
    -generic EN_QUEUE_DEPTH=0 \
    -generic EN_CLK_DIVIDER=0 \
    -generic EN_GRAY_COUNTER=0 \
    -generic CARRY_WIDTH=64
```

6.2 Parameter Sweep

For a carry-chain width sweep, iterate over CARRY_WIDTH values:

```
for W in 8 16 32 64 128 256 512; do
    vivado -mode batch -source syn_sweep.tcl -tclargs CARRY_WIDTH=$W
done
```

6.3 Cross-FUB Comparison

Enable all FUBs and use path groups to compare in a single run:

```
synth_design -top char_top   ;# All EN_* default to 1
report_timing -group GRP_NAND -max_paths 1
report_timing -group GRP_CARRY -max_paths 1
report_timing -group GRP_MULT -max_paths 1
# ... etc.
```

6.4 Cross-Technology Comparison

Synthesize the same RTL + SDC against different targets:

Target	Tool	Command
Xilinx Artix-7	Vivado	`set_property part xc7a200t...`
Xilinx UltraScale+	Vivado	`set_property part xcvu9p...`
Cyclone 10	Quartus	`set_global_assignment FAMILY "Cyclone 10 LP"`

Target	Tool	Command
Stratix 10	Quartus	`set_global_assignment FAMILY "Stratix 10"`
ASIC (TSMC 28nm)	DC/Genus	Point to `.db` / `.lib` library

The SDC is tool-agnostic (standard Tcl/SDC syntax) and works unchanged across all these flows.

---

7. Interpreting Results

7.1 Key Metrics

For each FUB at each parameter point, extract:

Metric	Source	Meaning
Setup slack	Timing report	How much margin remains (negative = fails)
Data path delay	Timing report	Actual combinational delay
Logic levels	Timing report	Number of LUT/gate levels
Resource usage	Utilization report	LUTs, FFs, DSPs, BRAMs consumed
Fmax	`1 / (period - slack)`	Maximum achievable frequency

7.2 Expected Trends

FUB	Parameter sweep	Expected trend
nand_chain	LEVELS 1-12	Linear delay increase per level
inverter_chain	COUNT 1-256	Linear delay, flattens when routing dominates
xor_tree	LEVELS 1-12	Similar to NAND but potentially fewer logic levels (better LUT packing)
carry_chain	WIDTH 8-512	Sub-linear delay (dedicated carry is fast)
multiplier_tree	WIDTH 8-32, TYPE 0-4	TYPE=0 uses DSP (fast), structural types use LUTs (slower)
mux_tree	LEVELS 1-12	~2 levels per LUT on 6-input LUT architectures
queue_depth	DEPTH 4-4096	Step changes at LUTRAM/BRAM/URAM boundaries

FUB	Parameter sweep	Expected trend
clock_divider_chain	STAGES 1-8	Linear per stage
gray_counter_chain	WIDTH 4-128	Dominated by XOR tree above ~32 bits

7.3 Normalizing Across Technologies

To compare across technologies, normalize to **delay per logic level**:

```
delay_per_level = data_path_delay / logic_levels
```

This removes the depth variable and isolates the per-gate (or per-LUT) delay, which is the true technology-dependent metric.

---

8. File Organization

```
rtl/syn/
  char_top.sdc           <-- This SDC file (constraints)
  SYNTHESIS_GUIDE.md      <-- This document

rtl/fub/                  <-- FUB RTL (one per characterization block)
  nand_chain.sv
  inverter_chain.sv
  xor_tree.sv
  carry_chain.sv
  multiplier_tree.sv
  mux_tree.sv
  queue_depth.sv
  clock_divider_chain.sv
  gray_counter_chain.sv

rtl/top/                  <-- Top-level wrappers
  char_top.sv             <-- Main synthesis target
  nand_chain_top.sv       <-- Standalone NAND chain wrapper

rtl/common/               <-- Shared dependencies (copied from
rtl/common/)
  counter_bin.sv
  counter_bingray.sv
  clock_divider.sv
  fifo_control.sv
  gaxi_fifo_sync.sv
  math_adder_half.sv
  math_adder_full.sv
  math_adder_carry_save.sv
  math_compressor_4to2.sv
  math_multiplier_dadda_tree_*.sv
  math_multiplier_wallace_tree_*.sv
  math_multiplier_wallace_tree_csa_*.sv
  math_multiplier_dadda_4to2_*.sv
  math_adder_han_carlson_*.sv
  math_prefix_cell.sv
  math_prefix_cell_gray.sv

rtl/filelists/            <-- Filelist for synthesis and simulation
  char_top.f
```

9. Quick Reference – Recommended First Runs

Run	Enable flags	Key parameter	Goal
Carry sweep	`EN_CARRY_CHAIN=1`, all others 0	`CARRY_WIDTH=8,16,32,64,128,256`	Map carry-chain speed vs. width
NAND depth	`EN_NAND_TREE=1`	`NAND_LEVELS=4,6,8,10,12`	Per-gate delay baseline
DSP vs. LUT	`EN_MULTIPLIER=1`	`MULT_TYPE=0` then `MULT_TYPE=1`, `WIDTH=16`	Quantify DSP advantage
Memory inference	`EN_QUEUE_DEPTH=1`	`FIFO_DEPTH=4,16,64,256,1024,4096`	Find LUTRAM/BRAM boundary
Full comparison	All enabled	Defaults	Cross-FUB comparison at one point

Version: 1.1 **Last Updated:** 2026-10-20 **Maintained By:** RTL Design Sherpa Project